

INFORME DE INSTALACIÓN DE SELENIUM EN VISUAL STUDIO CODE

1. Introducción

El presente informe describe el procedimiento realizado para instalar y preparar Selenium en el proyecto de la tienda en línea, utilizando Visual Studio Code como entorno de desarrollo.

La actividad se desarrolla desde el backend del proyecto y comprende la ubicación de la carpeta de Selenium, la creación de la carpeta correspondiente al módulo de login, el ingreso del código de prueba y su ejecución desde la terminal.

2. Objetivo general

Instalar y configurar Selenium en Visual Studio Code para realizar pruebas automatizadas sobre la aplicación web del proyecto.

3. Procedimiento realizado

Inicialmente se ingresa al proyecto desde Visual Studio Code y se accede a la carpeta correspondiente al backend. Desde esta ubicación se instala y organiza Selenium para trabajar con las pruebas automatizadas.

Posteriormente se ubica la carpeta de Selenium dentro del backend. Después se crea la carpeta destinada al proceso de login, en la cual se incorpora el código necesario para realizar la prueba automatizada.

Finalmente, desde la terminal se ejecuta el código de prueba. Después de unos segundos se obtiene el resultado de la ejecución, permitiendo comprobar el comportamiento del proceso automatizado.

4. Desarrollo de la actividad

El procedimiento se realizó siguiendo el orden indicado en el documento de la actividad: primero se trabaja en el backend y se instala Selenium; posteriormente se ubica la carpeta de Selenium; luego se crea la carpeta para login y se incorpora el código correspondiente; por último, se ejecuta el código desde la terminal para obtener el resultado.

5. Resultado

La ejecución de la prueba permite verificar el funcionamiento del proceso automatizado de login mediante Selenium. De esta forma, se deja preparado el entorno para continuar con la automatización de pruebas de la aplicación.

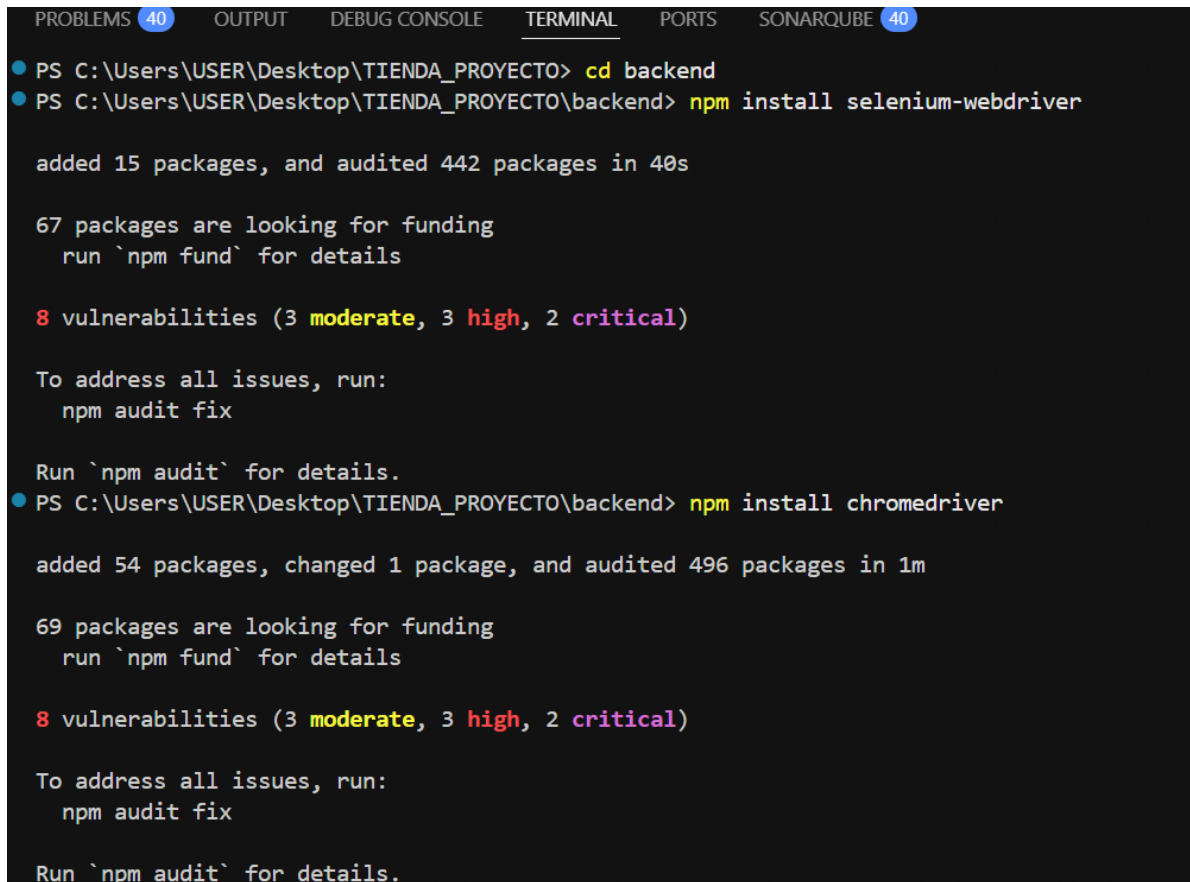
6. Conclusiones

La instalación de Selenium en Visual Studio Code permite disponer de una herramienta para automatizar pruebas sobre la aplicación web. El procedimiento realizado permitió organizar

Selenium dentro del backend, preparar el módulo de login y ejecutar el código de prueba desde la terminal.

La actividad constituye una base para continuar implementando pruebas automatizadas sobre las diferentes funcionalidades del proyecto de tienda en línea.

Imágenes de la actividad



```
PROBLEMS 40 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 40
PS C:\Users\USER\Desktop\TIENDA_PROYECTO> cd backend
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend> npm install selenium-webdriver

added 15 packages, and audited 442 packages in 40s

67 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (3 moderate, 3 high, 2 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend> npm install chromedriver

added 54 packages, changed 1 package, and audited 496 packages in 1m

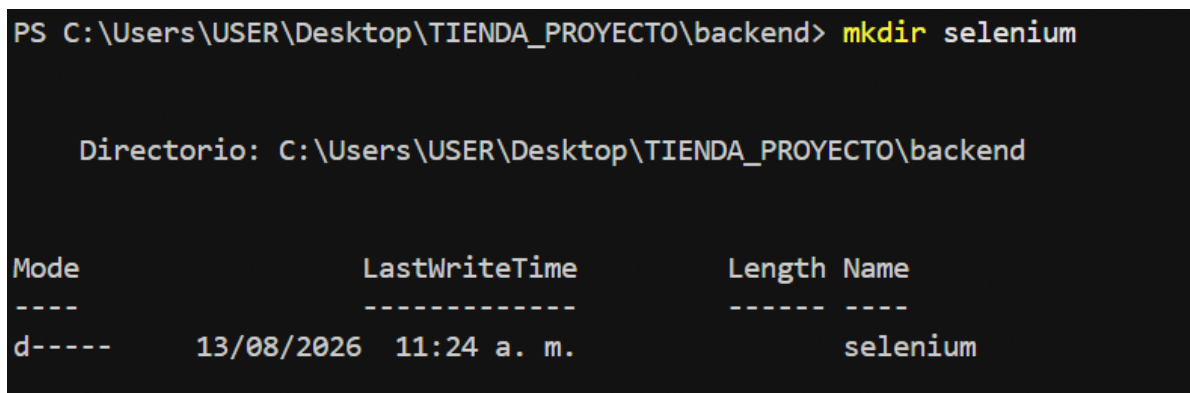
69 packages are looking for funding
  run `npm fund` for details

8 vulnerabilities (3 moderate, 3 high, 2 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
```

Imagen 1. Captura tomada del documento de la actividad.

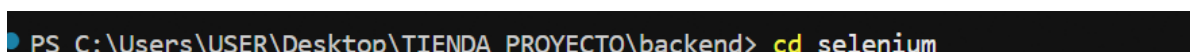


```
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend> mkdir selenium

Directorio: C:\Users\USER\Desktop\TIENDA_PROYECTO\backend

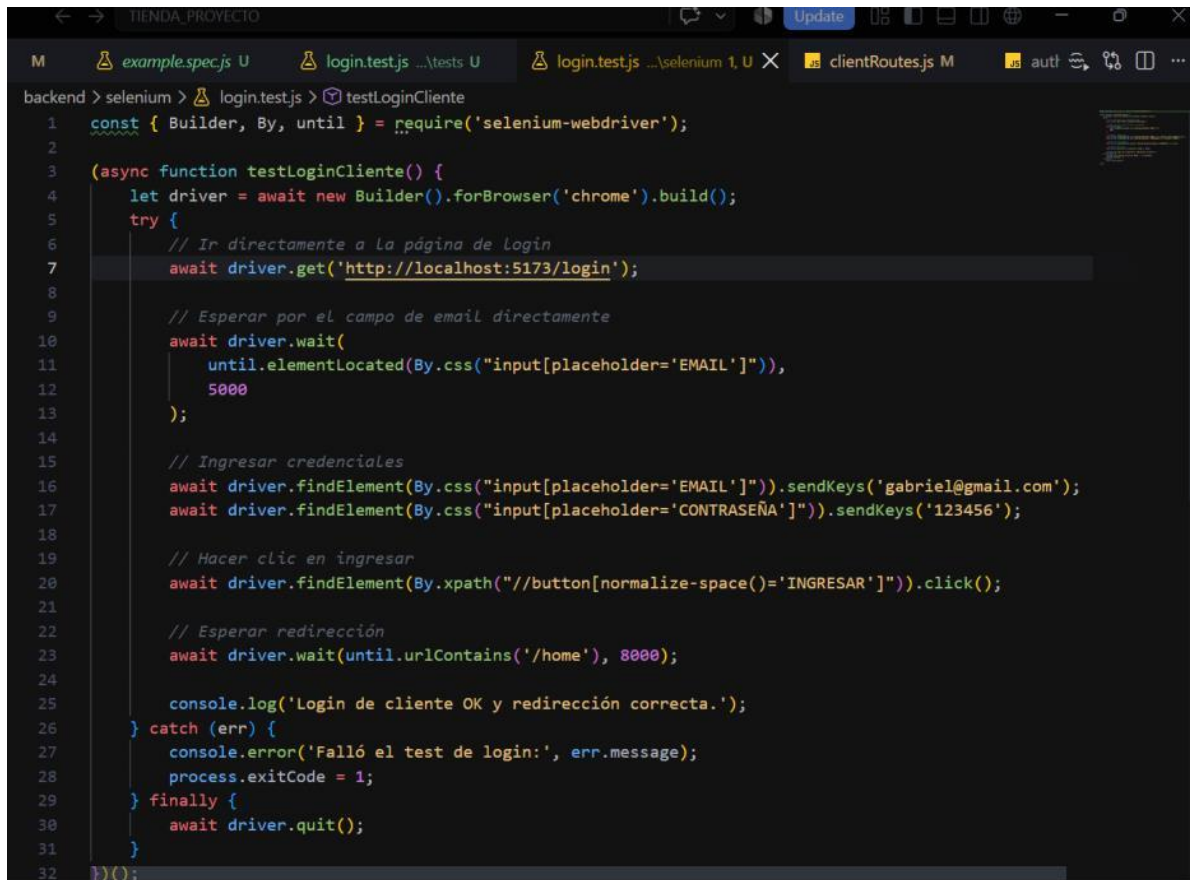
Mode                LastWriteTime         Length Name
----                -
d-----           13/08/2026  11:24 a. m.         selenium
```

Imagen 2. Captura tomada del documento de la actividad.



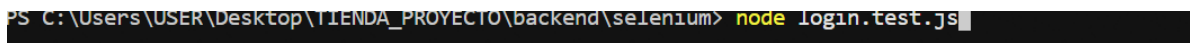
```
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend> cd selenium
```

Imagen 3. Captura tomada del documento de la actividad.



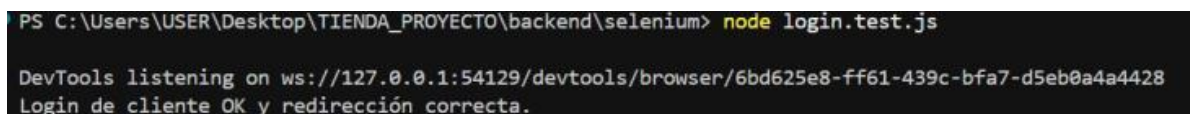
```
backend > selenium > login.test.js > testLoginCliente
1  const { Builder, By, until } = require('selenium-webdriver');
2
3  (async function testLoginCliente() {
4      let driver = await new Builder().forBrowser('chrome').build();
5      try {
6          // Ir directamente a la página de Login
7          await driver.get('http://localhost:5173/login');
8
9          // Esperar por el campo de email directamente
10         await driver.wait(
11             until.elementLocated(By.css("input[placeholder='EMAIL']")),
12             5000
13         );
14
15         // Ingresar credenciales
16         await driver.findElement(By.css("input[placeholder='EMAIL']")).sendKeys('gabriel@gmail.com');
17         await driver.findElement(By.css("input[placeholder='CONTRASEÑA']")).sendKeys('123456');
18
19         // Hacer clic en ingresar
20         await driver.findElement(By.xpath("//button[normalize-space()='INGRESAR']")).click();
21
22         // Esperar redirección
23         await driver.wait(until.urlContains('/home'), 8000);
24
25         console.log('Login de cliente OK y redirección correcta.');
```

Imagen 4. Captura tomada del documento de la actividad.



```
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend\selenium> node login.test.js
```

Imagen 5. Captura tomada del documento de la actividad.



```
PS C:\Users\USER\Desktop\TIENDA_PROYECTO\backend\selenium> node login.test.js
DevTools listening on ws://127.0.0.1:54129/devtools/browser/6bd625e8-ff61-439c-bfa7-d5eb0a4a4428
Login de cliente OK y redirección correcta.
```